

What is a tokenizer?

A **tokenizer** is a tool used in natural language processing (NLP) and AI models that **splits text into smaller pieces**, called **tokens**.

Why do we need it?

AI models (like transformers, GPT, BERT) do not understand raw human text — they only work with numbers (tensors). The tokenizer converts text → numbers in a consistent way.

What is a "token"?

A **token** depends on the tokenizer type:

Tokenizer Type	What is a token?
Word tokenizer	Words (e.g. "Hello", "world")
Subword tokenizer (BPE)	Sub-parts of words (e.g. "Hel", "lo")
Character tokenizer	Single characters (e.g. "H", "e")
Sentence tokenizer	Sentences

Modern models like **GPT** or **BERT** typically use **subword tokenizers** — this is more efficient and handles rare words better.

Example

If we tokenize:

```
"Tokenization is fun!"
```

One possible tokenization (BPE-based) might look like:

```
["Token", "ization", " is", " fun", "!" ]
```

Then each token is converted to an integer ID (so the model can process it):

```
[1532, 9874, 502, 1290, 93]
```

Main steps:

- 1. **Text** → **tokens**
- 2. **Tokens** → **token IDs (numbers)**
- 3. **Feed token IDs to model**

Understand the numbers or Tokens IDs:

This is exactly the *core idea* of tokenization!

Let me walk you through **why each token becomes a number**, and how the model understands it:

◆ Step 1: Why convert text to numbers?

Neural networks (like GPT) do not work with text — they only work with **vectors** (numbers). So we need a way to map each token to a number — this is called an **integer ID**.

◆ Step 2: What is the vocabulary?

- The tokenizer has a "vocabulary" — a list of *all possible tokens* it knows.
- Each token in the vocabulary has an **index** → this is the **token ID**.

Example:

Token	ID
"Token"	1532
"ization"	9874
" is"	502
" fun"	1290
"!"	93

◆ Step 3: What happens at runtime?

When you write:

```
Tokenization is fun!
```

The tokenizer breaks it into:

```
["Token", "ization", " is", " fun", "!" ]
```

Then it looks up each token in the vocabulary and replaces it with its ID:

```
[1532, 9874, 502, 1290, 93]
```

♦ Step 4: Why use subword tokens?

You might ask:

Why not just use "Tokenization" as one token?

Answer: Subword tokens (like "Token", "ization") allow the model to handle:

✅ Rare words ✅ Compound words ✅ Misspellings ✅ Many languages

This makes the model more flexible and efficient — otherwise it would need **one token per word**, which is too big!

♦ Step 5: What happens inside the model?

The model takes these token IDs (numbers):

```
[1532, 9874, 502, 1290, 93]
```

And uses them to look up **embedding vectors**:

```
token_id → embedding vector
```

Each ID maps to a **vector** (a big array of numbers), for example:

```
1532 → [0.12, -0.34, 0.98, ...] # 4096 numbers (if model size = 4096)
```

These embeddings are what the neural network actually processes.

Summary

Step	What happens
1	Text → tokens (subwords)
2	Tokens → token IDs (integers)
3	Token IDs → embeddings (vectors)
4	Model processes embeddings

If you want, I can also show:

✅ How to print the tokens & token IDs in Python ✅ How embeddings work internally ✅ Why different models (GPT-4o vs GPT-4 vs GPT-3.5) have different token vocabularies

How to use the tokenizer in Python of OpenAI API:

tokenization in the OpenAI API (for GPT-4, GPT-3.5, etc.). OpenAI models use a tokenizer called **tiktoken**.

❶ What is tiktoken?

tiktoken is the fast tokenizer used by OpenAI to split text into tokens for GPT models. Each model (GPT-3.5, GPT-4, GPT-4o) has its own tokenizer because the training data and vocabulary are slightly different.

- It's a **byte pair encoding (BPE)** tokenizer → works at subword level.
 - It efficiently compresses text into tokens.
 - Some words become 1 token, some get split into multiple tokens.
-

❷ Installing tiktoken

If you want to use the tokenizer locally:

```
pip install tiktoken
```

❸ Simple Example

```
import tiktoken

# Load the tokenizer for GPT-4o (you can use 'gpt-3.5-turbo' or 'gpt-4'
# too)
encoding = tiktoken.encoding_for_model("gpt-4o")

# Your text
text = "Tokenization is fun!"

# Convert text to tokens
tokens = encoding.encode(text)

# Print tokens (as numbers)
print(tokens)

# Convert tokens back to text
decoded_text = encoding.decode(tokens)
print(decoded_text)
```

4 How many tokens in my prompt?

```
num_tokens = len(encoding.encode(text))
print(f"Number of tokens: {num_tokens}")
```

5 Why is this important?

When you use the OpenAI API, **you pay per token**, and there is a **max token limit per model**:

Model	Max tokens
GPT-3.5-turbo	16,385
GPT-4	8,192
GPT-4o	128,000

6 Example — token count in a chat message

```
# For chat format (OpenAI API with messages), use tiktoken + helper
function:
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4o")

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Tell me about tokenization."}
]

# Approximate count: sum of tokens in each message content + formatting
overhead
num_tokens = sum([len(encoding.encode(msg["content"])) for msg in
messages])
print(f"Token count: {num_tokens}")
```

If you want, I can also show:

✓ Exact token calculation for full OpenAI chat messages ✓ Why "hello world" is fewer tokens than "Congratulations, you won a prize!" ✓ How to avoid hitting token limits in your apps

Tokenizers for LLMs:

- Each model **usually has its own tokenizer** or at least its own tokenizer configuration. Here's why:

Why does each model have its own tokenizer?

1. **Different vocabularies:** Each model is trained with a specific vocabulary — the set of tokens it knows. That means the tokenizer needs to match that vocabulary exactly.
2. **Different tokenization rules:** The way text is split into tokens (subwords, characters) may vary slightly depending on the model’s training corpus and design.
3. **Efficiency and model design:** Some models have special tokens or handle whitespace and punctuation differently. Their tokenizer is tailored for that.

Examples from OpenAI:

Model	Tokenizer used
GPT-3 (davinci)	GPT-3 tokenizer
GPT-3.5-turbo	GPT-3.5 tokenizer (slightly updated)
GPT-4	GPT-4 tokenizer (differs slightly)
GPT-4o (GPT-4 optimized)	GPT-4o tokenizer (custom for efficiency)

How to pick the right tokenizer?

If you use the OpenAI `tiktoken` Python library, you call:

```
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4o")
```

This automatically loads the **correct tokenizer** for that model.

What happens if you use the wrong tokenizer?

- The tokens won’t match what the model expects.
 - Your input could be broken differently.
 - Token counts might be off, leading to errors or inefficient use of tokens.
 - The model might not understand the input properly.
-

TL;DR:

Yes, each OpenAI model has its own tokenizer optimized for it, and you should always use the tokenizer specific to the model you're calling to get accurate tokenization and token counts.

Example to compare the tokenization of the same text between two OpenAI models — GPT-3.5-turbo and GPT-4o:

Sure! Let me give you a detailed Python example that **compares tokenization of the same text between two OpenAI models — GPT-3.5-turbo and GPT-4o** — side by side, including tokens, token IDs, and token counts.

```
import tiktoken

# The text to tokenize
text = "Tokenization is fun!"

# Load tokenizer for GPT-3.5-turbo
encoding_3_5 = tiktoken.encoding_for_model("gpt-3.5-turbo")

# Load tokenizer for GPT-4o
encoding_4o = tiktoken.encoding_for_model("gpt-4o")

# Tokenize the text using GPT-3.5 tokenizer
tokens_3_5 = encoding_3_5.encode(text)
decoded_3_5 = encoding_3_5.decode(tokens_3_5)

# Tokenize the text using GPT-4o tokenizer
tokens_4o = encoding_4o.encode(text)
decoded_4o = encoding_4o.decode(tokens_4o)

# Function to pretty-print tokens with their IDs and decoded text
def print_token_info(name, tokens, decoded):
    print(f"\n--- {name} ---")
    print("Token IDs:", tokens)
    print(f"Number of tokens: {len(tokens)}")
    print("Decoded text:", decoded)

print_token_info("GPT-3.5-turbo", tokens_3_5, decoded_3_5)
print_token_info("GPT-4o", tokens_4o, decoded_4o)

# Additionally, print the actual tokens (strings) by decoding token IDs
individually
def tokens_to_strings(encoding, token_ids):
    return [encoding.decode([tid]) for tid in token_ids]

print("\nToken strings for GPT-3.5-turbo:", tokens_to_strings(encoding_3_5,
tokens_3_5))
print("Token strings for GPT-4o:", tokens_to_strings(encoding_4o,
tokens_4o))
```

What this does:

- Tokenizes the exact same text using both GPT-3.5 and GPT-4o tokenizers.

- Prints token IDs (the numbers the model actually sees).
 - Prints the number of tokens used.
 - Prints the decoded text (should match original).
 - Prints the actual string tokens that correspond to each token ID.
-

Expected outcome

You'll see something like:

```
--- GPT-3.5-turbo ---  
Token IDs: [1234, 5678, 90, ...]  
Number of tokens: 5  
Decoded text: Tokenization is fun!  
  
--- GPT-4o ---  
Token IDs: [4321, 8765, 91, ...]  
Number of tokens: 6  
Decoded text: Tokenization is fun!  
  
Token strings for GPT-3.5-turbo: ['Token', 'ization', ' is', ' fun', '!']  
Token strings for GPT-4o: ['Token', 'ization', ' is', ' fun', '!']
```

Why might the token counts or IDs differ?

- Models have **different vocabularies**.
 - Tokenizers may split tokens slightly differently.
 - They can add or remove special tokens internally.
-

If you want, I can help you run this code or explain the tokenization differences in more detail!